



**Tribhuvan University
Faculty of Humanities and Social Science
Samarpan Academy**

**A PROJECT REPORT
ON
INTERNSHIP MANAGEMENT SYSTEM(IMS)**

**Submitted to
Department of Computer Application
Samarpan Academy**

In partial fulfillment of the requirements for the Bachelors in Computer Application

**Submitted by
Shushanta Dhungana
TU Registration number: 6-2-1200-13-2020
November 2025**

**Under the Supervision of
Niko Gurung**



Tribhuvan University
Faculty of Humanities and Social Science
Samarpan Academy

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by Shushanta Dhungana entitled “**Internship Management System**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

SIGNATURE

Niko Gurung

SUPERVISOR

Faculty of Humanities and Social Science

Dhumbarahi, Kathmandu



Tribhuvan University
Faculty of Humanities and Social Science
Samarpan Academy

LETTER OF APPROVAL

This is to certify that this project prepared by Shushanta Dhungana entitled “**Internship Management System**” in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Signature of Supervisor Mr. Niko Gurung Samarpan Academy Dhumbarahi, Kathmandu	Signature of HOD/ Coordinator Mr. Niko Gurung Coordinator Samarpan Academ
Signature of Internal Examiner Samarpan Academy	Signature of External Examiner

ABSTRACT

Inventory Management System using K-means Clustering for Customer Segmentation was developed to streamline business operations, enhance inventory visibility, and support data-driven decision-making. The system featured modules for dealer management, inventory tracking, purchase handling, invoice generation, party ledger creation, and customer record management. Customer segmentation was enabled through the K-means clustering algorithm, allowing intelligent grouping and analysis. Dynamic dashboards provided real-time insights into stock levels, transactions, and performance metrics, while a media center ensured efficient asset organization.

The platform was built using **FastAPI, Next.js, and MongoDB**, with RESTful APIs facilitating secure and efficient communication between services. The result was a responsive, modular, and user-friendly solution capable of managing inventory, financial records, and customer relationships effectively at scale.

Keywords: *Inventory Control & Management System, FastAPI, Next.js, MongoDB, REST API, K-means Clustering*

ACKNOWLEDGEMENT

I sincerely express my heartfelt gratitude to everyone who supported me throughout the completion of this project. I extend special thanks to **Er. Dhiraj Kumar Jha** for his invaluable guidance, constructive feedback, and continuous encouragement, which greatly contributed to the successful execution of this work. I am deeply grateful to my family for their constant love, motivation, and belief in me, and to my friends and college mates for their support, cooperation, and meaningful discussions during various stages of this project. I would also like to acknowledge the college library for providing essential resources and a conducive environment for research. To all who contributed directly or indirectly, I offer my sincere appreciation.

Sakar Dahal

TABLE OF CONTENT

SUPERVISOR’S RECOMMENDATION	i
LETTER OF APPROVAL	ii
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENT	v
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATION.....	ix
Chapter 1: Introduction.....	1
1.1 Introduction.....	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope and limitation	2
1.4.1 Scope of System.....	2
1.4.2 Limitation of Existing System	2
1.5 Development Methodology	2
1.6 Report Organization.....	3
Chapter 2: Background Study and Literature Review	4
2.1 Background Study.....	4
2.2 Literature Review.....	4
Chapter 3: System Analysis and Design	5
3.1 System Analysis	5
3.1.1 Requirement Analysis	5
3.1.2 Feasibility Study	7
3.1.3 Object Modeling	9

3.1.4	Dynamic Modeling	10
3.1.5	Process Modelling.....	12
3.2	System Design	13
3.2.1	Sequence Diagram	13
3.2.2	Component Diagram.....	15
3.2.3	Deployment Diagram.....	16
3.3	Algorithm Details.....	17
Chapter 4: Implementation and Testing		20
4.1	Implementation	20
4.1.1	Tools Used	20
4.1.2	Implementation Detail of Modules	21
4.2	Testing.....	22
4.2.1	Test Cases for Unit Testing	22
4.2.2	Test Cases for System Testing.....	23
Chapter 5: Conclusion and Future Recommendation		25
5.1	Conclusion	25
5.2	Future Enhancement	25
REFERENCES		26
APPENDICES		27

LIST OF FIGURES

Figure 1.1 Agile Methodology	3
Figure 3.1 Use case diagram	6
Figure 3.2 Gantt Chart	8
Figure 3.3 Class Diagram of Inventory Management System using K-means Clustering for Customer Segmentation	9
Figure 3.4 Sequence Diagram	10
Figure 3.5 Activity Diagram.....	12
Figure 3.6 Refined Sequence Diagram.....	13
Figure 3.7 Component Diagram	15
Figure 3.8 Deployment Diagram	16
Figure 3.9 Working Mechanism of K means Clustering	17
Figure 3.10 Elbow Curve for k Selection	19

LIST OF TABLES

Table 4.1 Implementation Detail of Modules	21
Table 4.2 Test Case 001 – Admin Login Module.....	22
Table 4.3 Test Case 001 Dealer Creation	23
Table 4.4 Test Case 002 - Add Product.....	24

LIST OF ABBREVIATION

API	: Application Programming Interface
CSS	: Cascading Style Sheets
DB	: Database
HTML	: Hypertext Markup Language
JS	: JavaScript
JWT	: JSON Web Token
MERN	: MongoDB, Express.js, React, Node.js
REST	: Representational State Transfer
SWR	: Stale While Revalidate
TC	: Test Case
UI	: User Interface
UX	: User Experience
VCS	: Version Control System

Chapter 1: Introduction

1.1 Introduction

The **Inventory Management System using K-means Clustering for Customer Segmentation** is a comprehensive solution designed to manage products, stock levels, dealers, and financial records efficiently. It enables model-based product tracking, category classification, purchase management, invoice generation, and party ledger maintenance. A centralized media center allows image reuse, and customer data is analyzed using the K-means clustering algorithm for effective segmentation. Built with **FastAPI, Next.js, and MongoDB**, the system provides real-time dashboards, reports, and search functionalities, ensuring smooth inventory operations and informed business decisions.

1.2 Problem Statement

Businesses often face challenges in managing inventory efficiently, tracking stock levels, maintaining accurate dealer and customer records, and monitoring financial transactions. Manual or fragmented systems lead to errors, stock-outs, overstocking, delayed invoicing, and difficulty in analyzing customer behavior. There is a need for a unified system that can streamline product management, automate stock updates, handle dealer and party ledgers, and provide real-time insights through dashboards and reports. Additionally, leveraging data analytics, such as customer segmentation, can help businesses make informed decisions and optimize operations.

1.3 Objectives

The objectives of the proposed system are:

- To efficiently manage dealer information, including company details, contact persons, payment terms, and transaction history.
- To maintain accurate and up-to-date records of stock supplied by each dealer, ensuring seamless inventory tracking.
- To automate party ledger management, tracking dues, payments, and outstanding balances for dealers.
- To streamline purchase and invoice generation processes linked to dealer transactions.

1.4 Scope and limitation

This section defines the boundaries of the Inventory Management System using K-means Clustering for Customer Segmentation, outlining its key functionalities in inventory and dealer management. It also highlights the system's constraints, such as reliance on manual entries and limited support for multi-branch operations.

1.4.1 Scope of System

The Inventory Management System using K-means Clustering for Customer Segmentation is designed to efficiently manage products, stock levels, dealers, and financial records within a business. It covers dealer management, model-based product tracking, category classification, purchase handling, invoice generation, and party ledger maintenance. The system also includes a centralized media center for image management and implements customer segmentation using the K-means clustering algorithm. Real-time dashboards and reports provide insights into stock status, financial transactions, and dealer performance, supporting data-driven decision-making and streamlined inventory operations.

1.4.2 Limitation of Existing System

- Certain financial transactions require manual entry.
- Analytical features are limited to basic clustering; no advanced forecasting.
- System performance depends on accurate and consistent data entry.
- Supports only a single organizational framework; no multi-branch synchronization.

1.5 Development Methodology

The Inventory Management System using K-means Clustering for Customer Segmentation was developed using the **Agile methodology**, emphasizing iterative development, continuous feedback, and incremental improvements. The system was built in phases, starting with requirement gathering, followed by system analysis, design, implementation, testing, and deployment. FastAPI was used for backend development, Next.js for the frontend, and MongoDB for the database, ensuring a modular, scalable, and maintainable architecture. This approach allowed for flexibility in incorporating changes, efficient progress tracking, and timely resolution of issues throughout the project lifecycle.

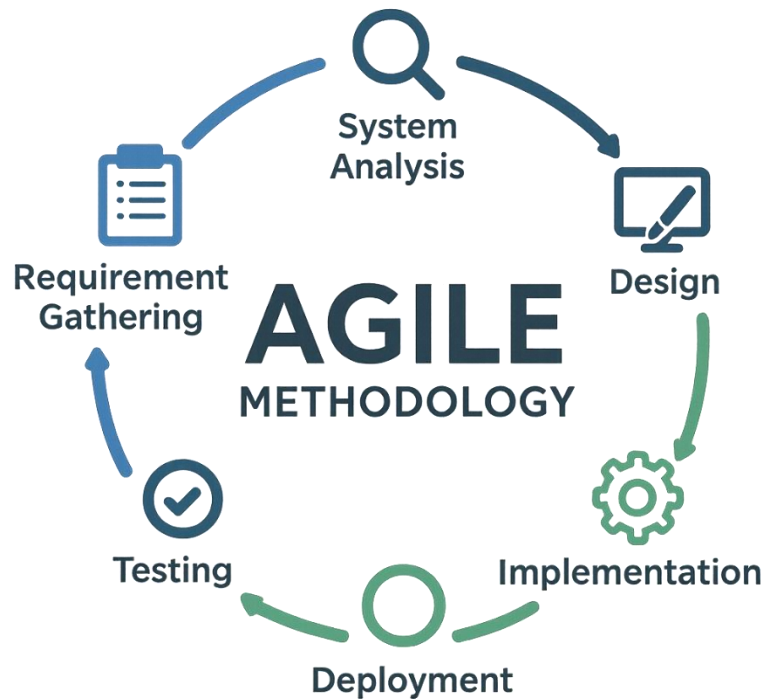


Figure 1.1 Agile Methodology

1.6 Report Organization

This report is organized into five chapters:

1. **Chapter 1 – Introduction:** Overview of the project, objectives, scope, and limitations.
2. **Chapter 2 – Background Study:** Analysis of existing systems and their pros and cons.
3. **Chapter 3 – System Analysis and Design:** Requirement analysis, feasibility study, and system design.
4. **Chapter 4 – Implementation and Testing:** System implementation, testing, and debugging details.
5. **Chapter 5 – Conclusion and Future Work:** Summary, limitations, and potential enhancements.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Inventory management is a key function in businesses, ensuring optimal stock levels, smooth operations, and effective management of dealers and customers. It involves processes such as tracking stock, managing purchases, maintaining ledgers, and analyzing customer and product data. Traditional manual methods or basic software often result in errors, stock discrepancies, delayed invoicing, and inefficient transaction tracking. Advanced inventory management systems aim to automate these processes, providing real-time updates, accurate stock control, and data-driven insights.

The Inventory Management System builds on these concepts by integrating dealer management, model-based product tracking, category classification, purchase and invoice handling, party ledger maintenance, and customer segmentation using K-means clustering. This ensures a unified, scalable, and user-friendly platform that improves operational efficiency, minimizes errors, and supports informed business decisions.

2.2 Literature Review

Several studies and projects have contributed to the development of effective inventory management systems. Multi-agent based inventory systems demonstrate how agent autonomy and cooperation can streamline inventory control, reduce stock levels, and enable seamless coordination among enterprises [1]. These ideas inspired the automated stock and dealer management features in the system, minimizing manual errors and improving workflow.

Web-based inventory systems emphasize centralized management, accessibility, and operational efficiency [2]. This influenced the development of a responsive, user-friendly interface in the system using FastAPI and Next.js, enabling efficient real-time stock management. The K-means clustering algorithm, widely applied in data segmentation and analysis, guided the design of system's customer segmentation feature, helping categorize customers for better decision-making and targeted management [3].

Additionally, integrating invoicing with analytics in small and micro enterprises demonstrates the importance of automated invoice generation, sales tracking, and financial management [4]. This informed the ledger and invoice modules in the system, ensuring accurate tracking of dues, payments, and financial summaries.

Chapter 3: System Analysis and Design

3.1 System Analysis

System analysis focuses on understanding the requirements, processes, and functionalities needed for the **Inventory Management System using K-means Clustering for Customer Segmentation**. It identifies what the system should do, how it should perform, and the constraints under which it will operate.

3.1.1 Requirement Analysis

Both functional and non-functional requirements were carefully examined and documented to serve as a foundation for system design and implementation. Requirements are generally split into two types:

- i. Functional Requirements
- ii. Non-Functional Requirements

i Functional Requirements

Functional requirements define the specific behaviors and functions of the system.

The key functional requirements of SYSTEM include:

- **Dealer Management** – Add, update, and delete dealer information; track dealer transactions and generate party ledger.
- **Product Management** – Add new products or update stock for existing models; manage product categories and specifications.
- **Stock Management** – Maintain real-time stock levels, track stock history, and generate low-stock alerts.
- **Media Center** – Upload, reuse, and manage product images with tagging and usage tracking.
- **Purchase & Invoice Management** – Record purchases, generate invoices, and track payments.
- **Customer Management & Segmentation** – Store customer data and perform segmentation using **K-means clustering**.
- **Reporting & Dashboard** – Provide stock summaries, financial metrics, and activity reports.

ii Non-Functional Requirements

Non-functional requirements define how the system performs and constraints under which it operates. For system, they include:

- **Performance** – Real-time updates for stock and ledger entries.
- **Scalability** – Ability to handle increasing products, dealers, and customers.
- **Security** – Secure API endpoints with Rate limiting, caching, token-based authentication.
- **Reliability** – System uptime, error handling, and data integrity.
- **Usability** – Intuitive and responsive web interface for ease of operation.

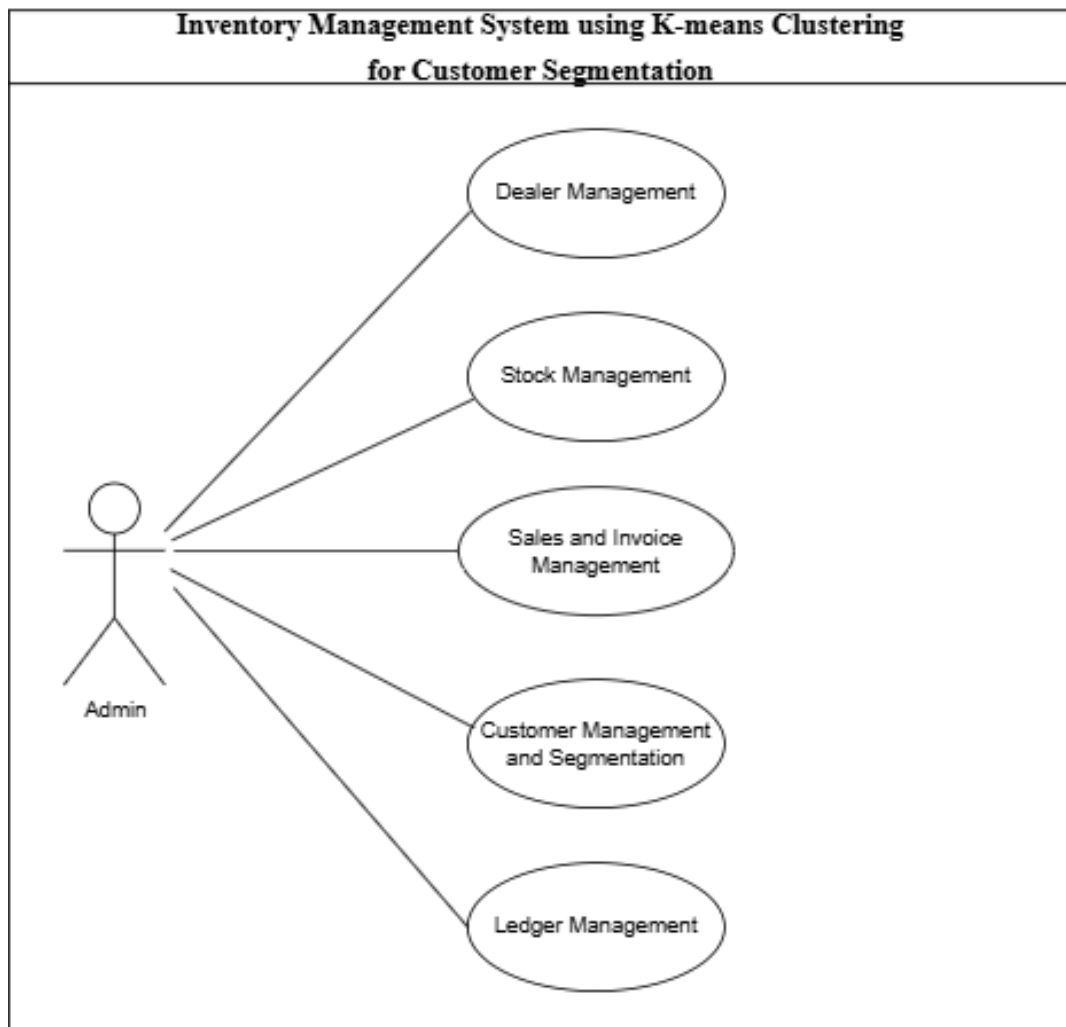


Figure 3.1 Use case diagram

The **admin** interacts with the system to manage all core aspects of inventory operations.

- Starting with **dealer management**, the admin maintains dealer records and transactions.
- **Stock management** ensures that product levels are always updated and monitored.
- Through **sales and invoice management**, the admin can create invoices and track sales seamlessly.
- Customer data is stored and analyzed via **customer management and segmentation**, improving decision-making with clustering techniques.
- Finally, **ledger management** ensures financial integrity by tracking dues, payments, and outstanding balances.

3.1.2 Feasibility Study

i. Technical Feasibility

The system is technically feasible because it uses modern, widely adopted technologies:

- **Backend:** FastAPI (lightweight, scalable, and high-performance Python framework).
- **Frontend:** Next.js (React-based, providing server-side rendering and optimized performance).
- **Database:** MongoDB (flexible schema design suitable for product and stock management).
- **Deployment:** Containerized using docker, ensuring flawless deployments to cloud platforms.

ii. Operational Feasibility

The system addresses real business problems faced by Inventory Management System using K-means Clustering for Customer Segmentation, such as:

- Managing dealer and supplier records.
- Tracking stock levels with model-based product management.
- Maintaining financial records through a party ledger.
- Reducing manual errors in invoice generation and stock updates.
- Providing real-time insights through dashboards and reports.

iii. Economic Feasibility

- **Development Costs:** Minimized by using open-source frameworks (FastAPI, Next.js, MongoDB).
- **Infrastructure Costs:** Controlled through cloud deployment on Google Cloud with scalable plans.
- **Maintenance Costs:** Reduced due to modular design and reusable components (e.g., Media Center).

iv. Schedule Feasibility

The system was developed using the Agile methodology, ensuring timely progress and flexibility:

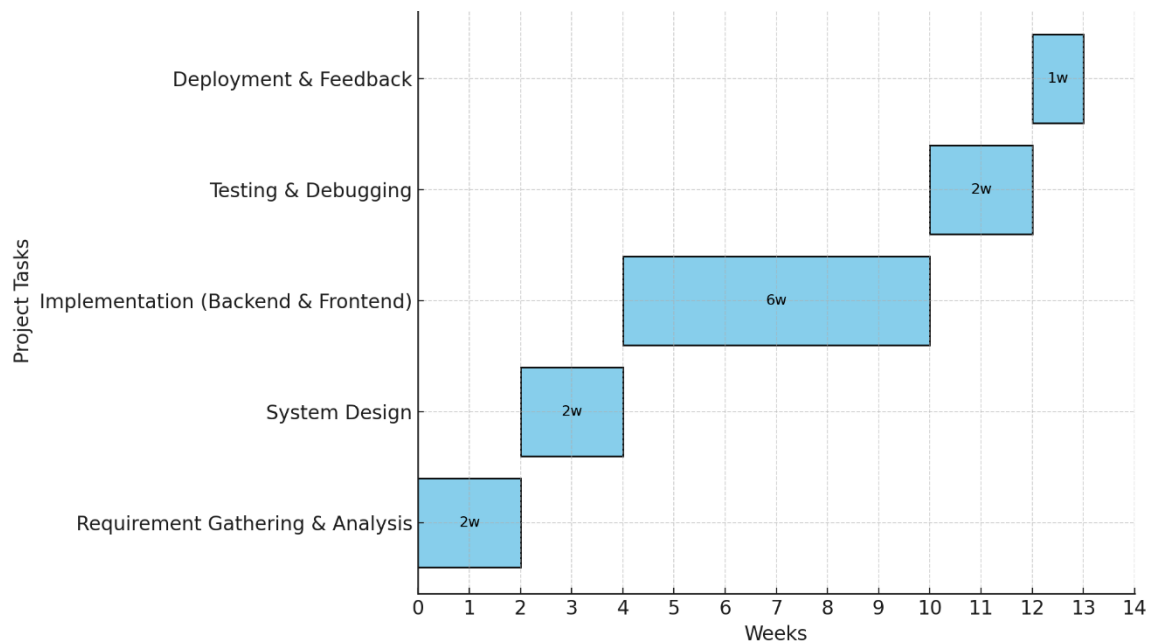


Figure 3.2 Gantt Chart

- **Requirement Gathering & Analysis** → 2 weeks
- **System Design** → 2 weeks
- **Implementation (Backend & Frontend)** → 6 weeks
- **Testing & Debugging** → 2 weeks
- **Deployment & Feedback** → 1 week

3.1.3 Object Modeling

The **class diagram** of the system represents the core entities of the inventory and dealer management system and how they interact. The **Dealer** class stores information about suppliers, including their name, phone number, GST number, and payment terms. Each dealer can supply multiple products, represented by the **Product** class, which contains details like model number, dealer price, and stock levels. Products are grouped into categories, managed by the **Category** class, such as TVs, Mobiles, or Refrigerators, allowing the system to organize items efficiently. Financial transactions with dealers are tracked using the **Ledger** class, which records dues, payments, and invoice references, and each dealer can have multiple ledger entries. Product images or media are handled by the **Media** class, and a many-to-many relationship exists between products and media, meaning a product can have multiple images, and a single image can be associated with multiple products. Overall, this class diagram defines the structural blueprint of the system, showing how dealers, products, categories, financial records, and media interconnect.

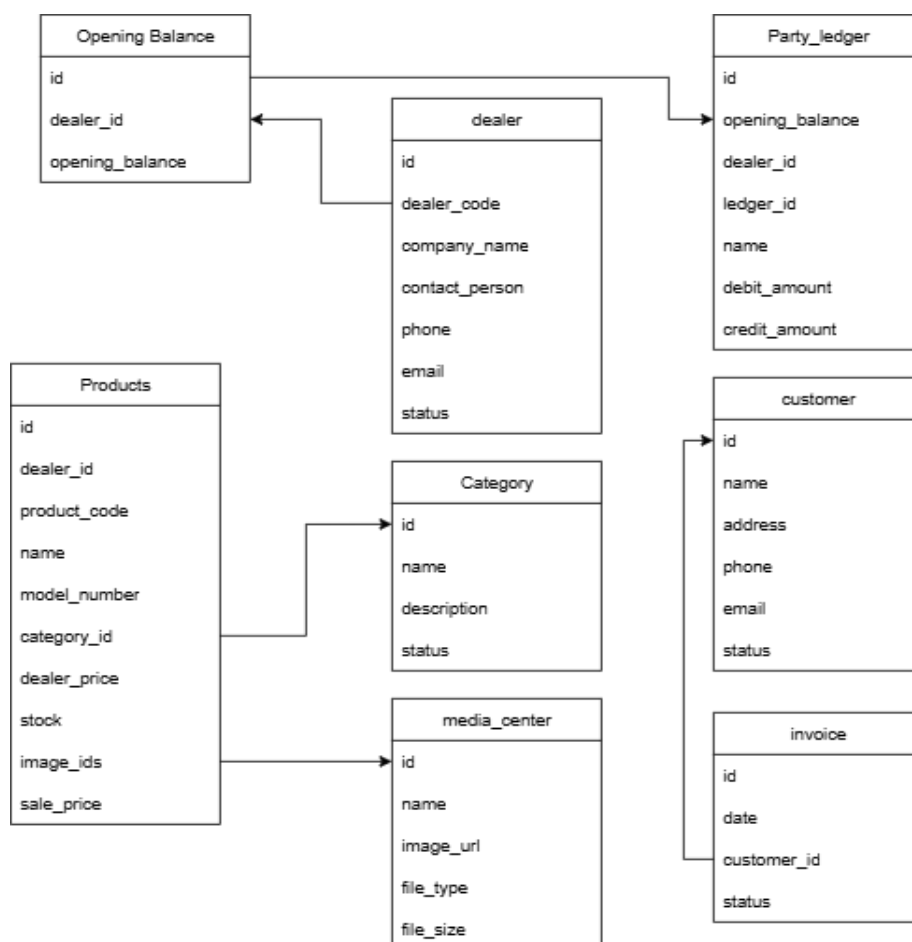


Figure 3.3 Class Diagram of Inventory Management System using K-means Clustering for Customer Segmentation

3.1.4 Dynamic Modeling

Dynamic modeling refers to the process of creating a representation of a system that captures how it changes over time. This involves understanding the behavior of a system through its interactions and the flow of information or resources, typically using various modeling techniques to depict dynamic aspects such as processes, events, and states. Here are some key points about dynamic modeling:

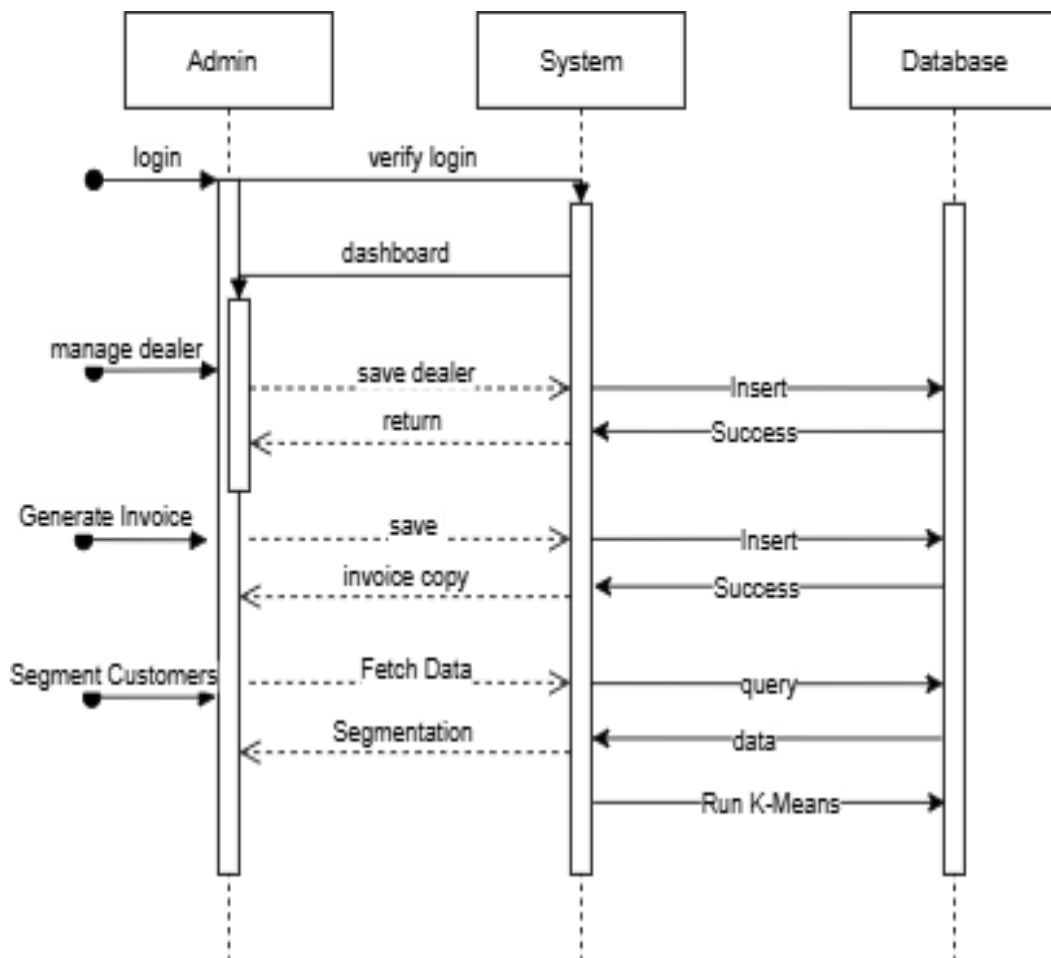


Figure 3.4 Sequence Diagram

The sequence diagram for the **Inventory Management System using K-means Clustering for Customer Segmentation** illustrates how the single administrator interacts with the system and how the system communicates with the underlying database to perform different operations. The diagram highlights the step-by-step workflow of requests, processing, and responses.

Login Process

- The Admin initiates the login by sending credentials to the system.
- The system verifies the credentials by querying the database.
- Upon successful authentication, the system provides the Admin with access to the dashboard.

Inventory Management

- When the Admin adds a new product, the request is sent to the system.
- The system forwards the product details to the database for storage.
- Once the operation is successful, the system responds back with an updated product list to the Admin.

Dealer Management

- The Admin can add or manage dealers by sending dealer information.
- The system saves these details into the database.
- A confirmation is sent back, and the updated dealer list is shown to the Admin.

Invoice Generation

- The Admin creates invoices for transactions.
- The system stores invoice records in the database.
- The Admin receives a copy of the generated invoice for reference.

Customer Segmentation

- To analyze customer behavior, the Admin requests segmentation.
- The system fetches customer data from the database and applies the **K-means clustering algorithm**.
- Segmentation results are then presented to the Admin for insights.

Reports and Analytics

- The Admin requests reports on stock levels, sales, or dealers.
- The system queries the database and processes the required data.
- The generated report is displayed to the Admin in a simplified format.

3.1.5 Process Modelling

Process modelling is a crucial step in system design as it visually represents the flow of activities, decisions, and interactions within a system. In the case of the system, activity diagrams are used to model how the administrator interacts with different system modules, from login to logout. These diagrams capture the dynamic behavior of the system and provide a clear understanding of sequential and parallel workflows.

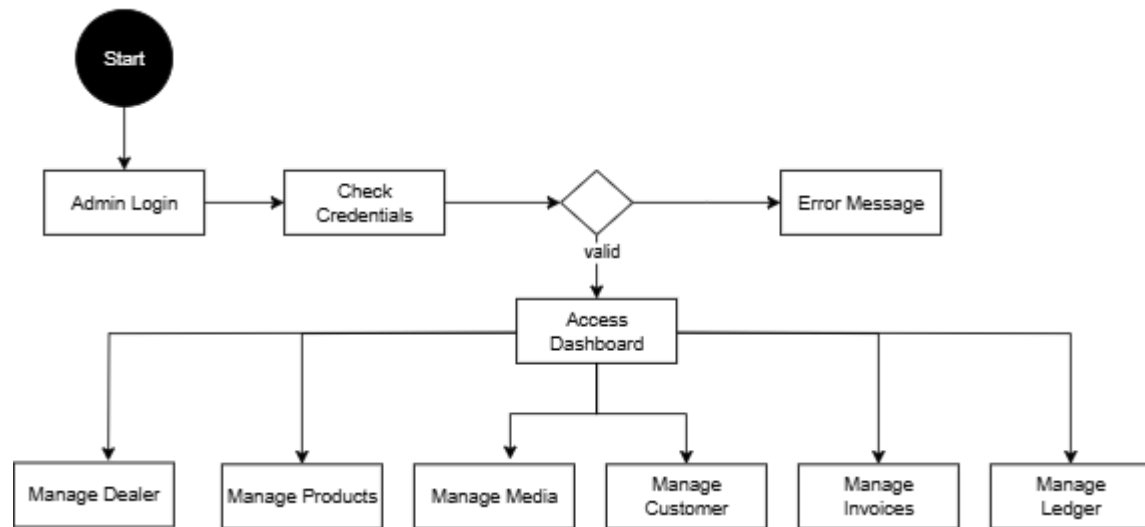


Figure 3.5 Activity Diagram

The activity diagram of the system begins with the administrator logging into the system. After successful authentication, the administrator gains access to the dashboard and selects from various operations such as managing inventory, managing dealers, generating invoices, or viewing reports and customer segmentation analytics. Each of these activities involves database interaction for storing or retrieving information. The process flow includes decision points, such as handling invalid login attempts, and ensures feedback is provided after every operation, such as success confirmations, invoice generation, or report display. Finally, the workflow ends when the administrator logs out of the system.

3.2 System Design

System design defines the overall structure, interactions, and deployment of the Inventory Management System using K-means Clustering for Customer Segmentation. It transforms the requirements into well-structured models and diagrams that represent the functional, behavioral, and physical aspects of the system. The design phase ensures that all modules, components, and workflows are properly visualized for accurate implementation.

3.2.1 Sequence Diagram

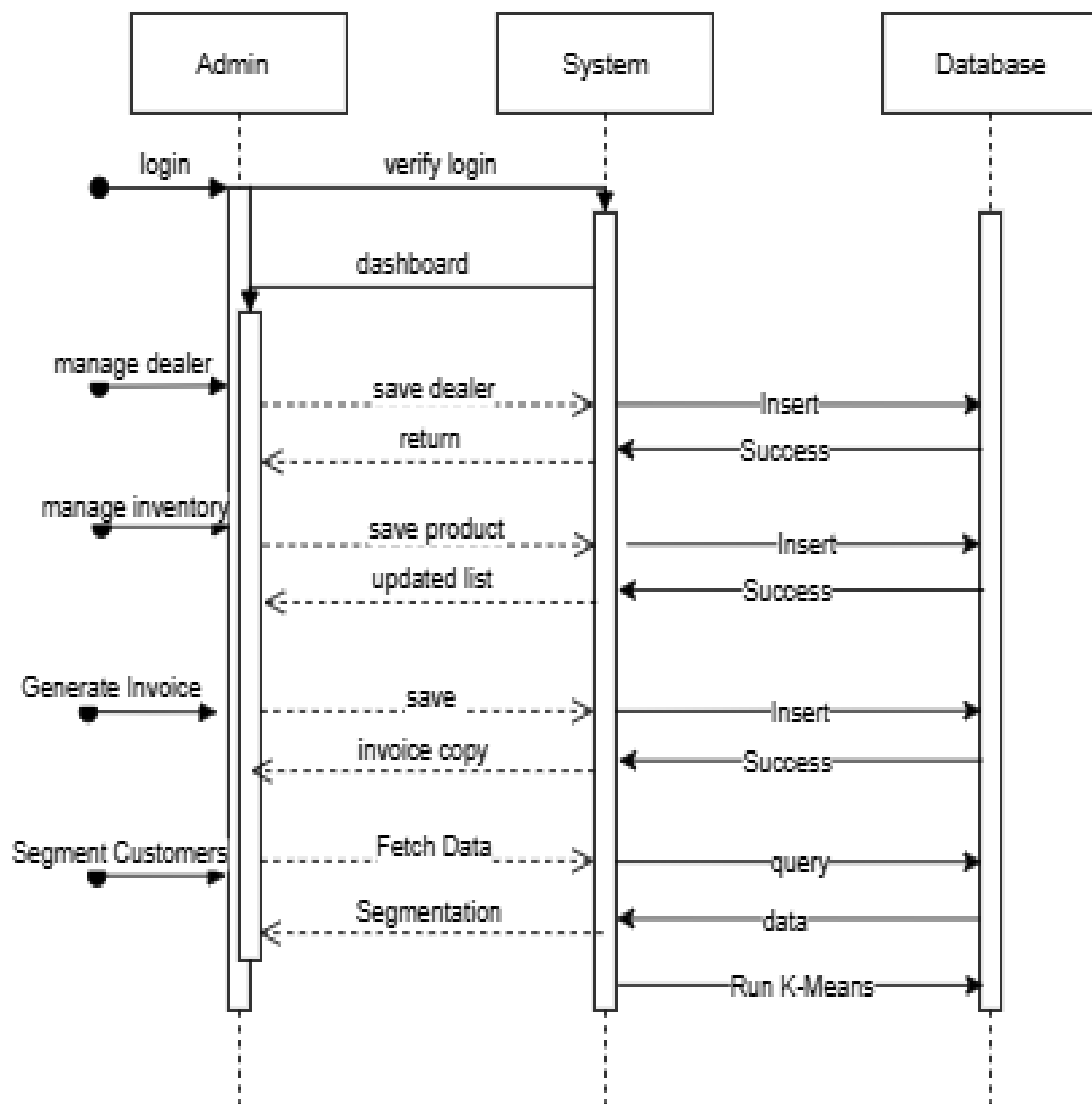


Figure 3.6 Refined Sequence Diagram

The refined sequence diagram for the system provides a detailed view of the interactions between the **Admin (actor)**, the **System (controller)**, and the **Database (data store)**. It ensures that all major operations are covered with precise workflows.

Login Authentication

- The Admin enters credentials.
- The System validates credentials by checking with the secret stored.
- If valid, the Admin is redirected to the dashboard; otherwise, an error is shown.

Inventory Management

- Admin requests to add or update a product.
- The System receives the request and forwards it to the Database.
- Once saved, the System confirms success and shows the updated inventory list to the Admin.

Dealer Management

- Admin submits dealer details for creation or modification.
- The System processes the request and updates the Database.
- Confirmation and updated dealer records are displayed back to the Admin.

Invoice Generation

- Admin initiates invoice creation by selecting customer and product details.
- The System generates an invoice, stores it in the Database, and retrieves a copy.
- The finalized invoice is displayed to the Admin.

Customer Segmentation (K-means Analytics)

- Admin requests customer segmentation.
- The System fetches customer data from the Database.
- K-means clustering algorithm is applied to categorize customers into groups.
- Results are returned and displayed as segmented clusters.

Reporting & Ledger Management

- Admin requests a report (e.g., sales, stock levels, dealer activity).
- The System queries the Database and processes the data.
- A structured report or ledger is generated and displayed.

3.2.2 Component Diagram

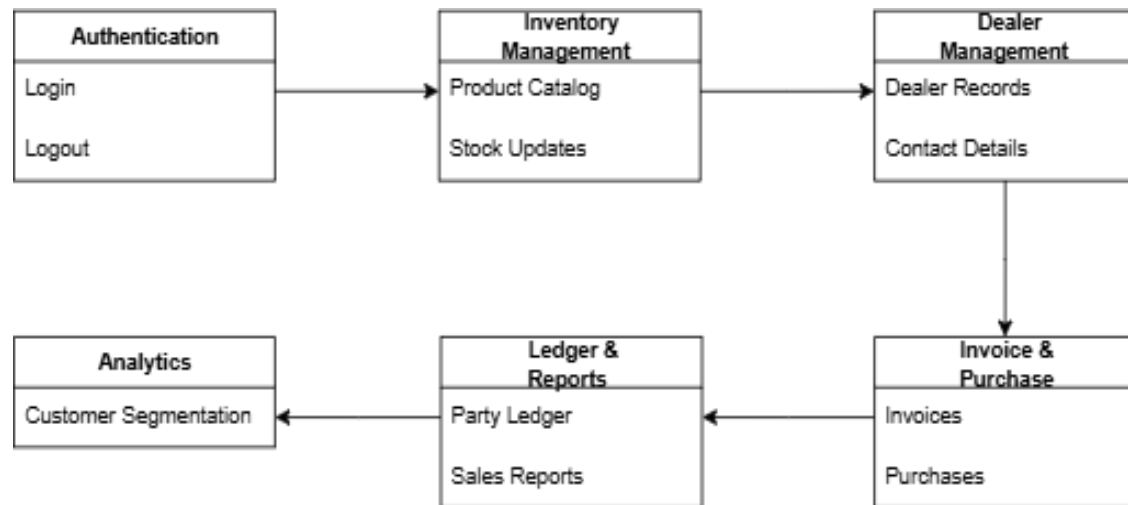


Figure 3.7 Component Diagram

The component diagram of the system illustrates the high-level structure of the system by showing its main software modules and their interactions. The **User Interface component** provides the front-end screens where users interact with the system, while the **Application/Controller component** acts as a bridge, handling requests and passing them to the backend. The **Business Logic/Service component** contains the core functionality of the system, enforcing rules for managing dealers, products, categories, ledgers, and media. Data persistence is handled by the **Database component**, which stores and retrieves information as required. This layered structure ensures a clear separation of concerns, improves maintainability, and highlights how different parts of system collaborate to provide complete functionality.

3.2.3 Deployment Diagram

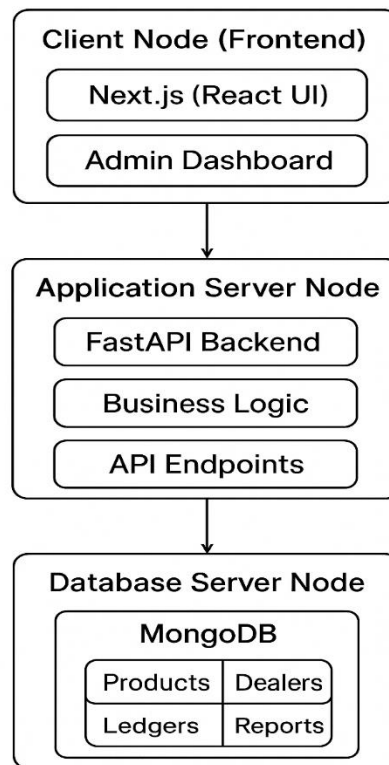


Figure 3.8 Deployment Diagram

This deployment diagram represents a **three-tier architecture** for the Inventory Management System using K-means Clustering for Customer Segmentation. Each node (box) corresponds to a hardware or server environment, and inside each node are the software components deployed on it. The arrows show communication paths between these nodes.

Client Node (Frontend)

- Contains the **Next.js (React UI)** and the **Admin Dashboard**.
- This is the entry point where users and administrators interact with the system.
- The frontend communicates with the backend using **HTTPS REST API calls**, ensuring secure communication.

Application Server Node

- Hosts the **FastAPI Backend**, which contains the **Business Logic** and **API Endpoints**.
- This is the core of the system where requests from the frontend are processed, business rules are applied, and data is prepared for users.

- The application server communicates with the database to fetch and update records as needed.

Database Server Node

- Runs **MongoDB**, which stores persistent data.
- Collections include **Products, Dealers, Ledgers, and Reports**, covering all critical aspects of the inventory and dealer management system.
- The database is accessed only by the application server, not directly by the client, ensuring data security.

Communication Flow

- The **Client Node** → **Application Server** communication happens via **HTTPS REST API calls**.
- The **Application Server** → **Database Server** communication happens through a **database connection** (MongoDB driver).

3.3 Algorithm Details

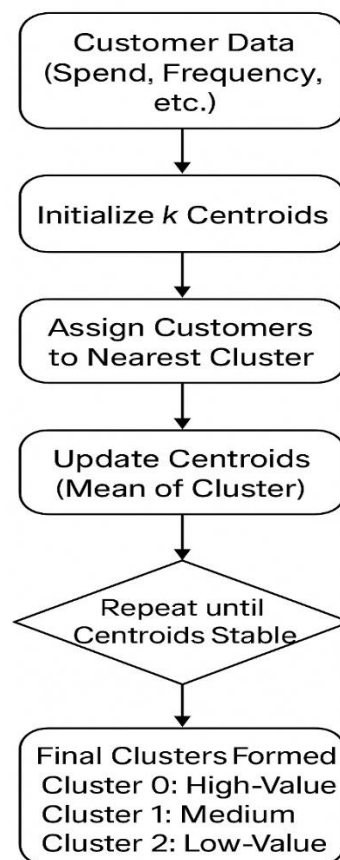


Figure 3.9 Working Mechanism of K means Clustering

This System's Customer Segmentation employs an exclusive version of the K-Means clustering technique for autonomously categorizing consumers according to their shopping habits. The objective is to assist businesses in identifying trends within their purchasing behavior for making informed strategic choices regarding marketing efforts, customer retention initiatives, and financial transactions.

Starting off involves collecting organized data points related to every client's spending habits, including overall total spend, number of completed invoices, average order value, and recency of purchase. The current focus of the system in cluster analysis involves examining only total spending and number of invoices as they effectively reflect key aspects of customer worthiness and interaction levels.

After preparing the data, the algorithm categorizes every client into one out of three groups initially set as k equals 3. Often these groups signify categories like high-value repeat customers, moderate-value regular customers, and low-spend occasional buyers. Following the clustering procedure, each client receives an integer-based cluster identifier (e. g. , 0 for VIPs, 1 for Active customers) subsequently translated into understandable descriptive terms like "High Value Champions" or "Active Growing Clients".

The K-means method undergoes successive adjustments for optimization. Firstly, it employs the K-Means++ method to randomly select k starting points called centroids. These initial positions aim to distribute data evenly across clusters effectively. Every client receives an immediate placement near its closest representative within the data's spatial framework. Following completion of this task cycle, new centers are determined through an average calculation of customer locations within every group formed. The task—repetitive update process persists till cluster centers settle down, indicating successful clustering completion. A distinct hierarchical arrangement emerges which facilitates precise strategic planning by businesses.

In this framework, employing k equals three aligns well with practical application insights. Retail customer profiling usually focuses on categorizing shoppers into three distinct groups based on their spending habits: frequent spenders who make many purchases, average consumers who occasionally shop, and infrequent purchasers. Ensuring clarity in segmentation by having three distinct groups guarantees it is both useful for action and consistent with typical business strategies while preventing overcomplicated classifications.

Determining the Value of k

Determining an appropriate value for k in the K-Means clustering algorithm is essential for forming meaningful customer groups. In this system, the selection process is guided by the Elbow Method, a standard evaluation technique that examines how clustering performance changes as the number of clusters increases. The Elbow Method focuses on inertia, which is the total squared distance between each data point and its assigned centroid. As the number of clusters grows, inertia decreases because the data is divided into more specific groups. However, this improvement does not continue at the same rate.

By conceptually observing inertia across increasing values of k , a pattern becomes clear. The decrease in inertia from k equal to 2 to k equal to 3 is relatively large, showing that introducing a third cluster adds meaningful separation in customer behavior. When moving from k equal to 3 to k equal to 4, the reduction becomes noticeably smaller. This point where the rate of improvement slows is known as the elbow. It indicates that additional clusters provide limited benefit and may lead to unnecessary fragmentation.

Based on this evaluation process, k equal to 3 corresponds to the elbow position and represents a balanced choice that maintains both clustering quality and practical interpretability for customer segmentation in the System.

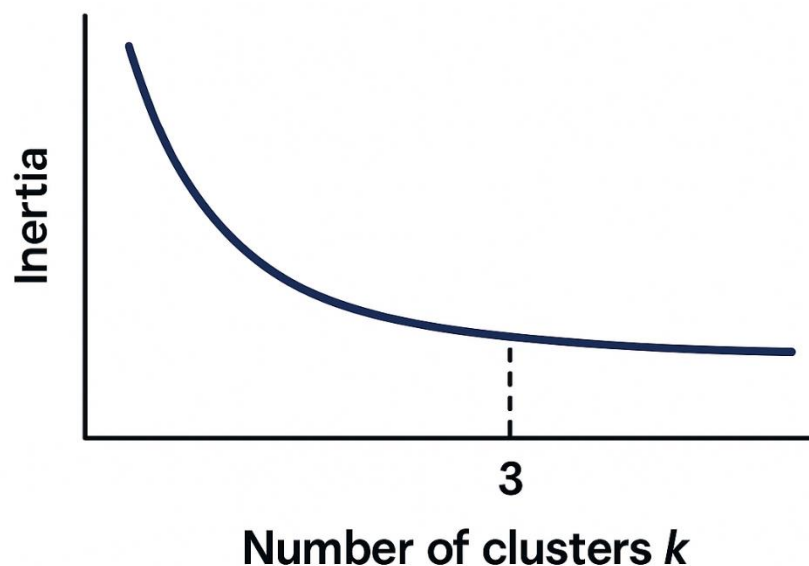


Figure 3.10 Elbow Curve for k Selection

Chapter 4: Implementation and Testing

4.1 Implementation

The system was developed using the **Agile development methodology**, ensuring iterative progress and continuous feedback throughout the project lifecycle. Development was carried out in short sprints, where features were planned, implemented, tested, and refined incrementally. This allowed for flexibility in adapting to evolving requirements and faster resolution of issues. Key modules such as authentication, product and dealer management, ledger handling, customer segmentation service, and reporting—were built and improved step by step. Regular testing and review ensured that each component met performance, scalability, and usability standards. This iterative approach ultimately led to a more robust, user-centered solution aligned with project goals.

4.1.1 Tools Used

The following tools and technologies were used for the design, development, and deployment of the system:

- **Next.js:** A React-based framework used for building the frontend interface and admin dashboard. It provides server-side rendering, API routes, and static site generation, which contribute to performance optimization, SEO-friendliness, and flexible deployment.
- **FastAPI (Python):** A modern, high-performance web framework for building the backend services. It was used to implement business logic, REST API endpoints, and integration with the database. Its speed and built-in validation made backend development efficient and reliable.
- **MongoDB:** A NoSQL database used for storing structured and semi-structured data, such as product information, dealer records, ledger transactions, and customer segmentation results. Its flexibility and scalability made it suitable for handling growing business data.
- **Cloudinary:** A cloud-based media management service used to store and deliver images, product photos, and other media assets. It provides efficient image optimization, transformations, and CDN delivery to improve frontend performance.
- **Google Cloud Platform (GCP):** Used for hosting and deploying the system. GCP services ensured scalability, reliability, and security of the application in production, supporting both the backend and frontend deployments.

4.1.2 Implementation Detail of Modules

Table 4.1 Implementation Detail of Modules

Code	Module Name	Description
M1	Login Module	Authenticates admin for secure system access.
M2	Dealer Module	Manages dealer details such as name, contact, GST, and payment terms.
M3	Product Module	Handles product info like model, pricing, stock levels, and category mapping.
M4	Category	Organizes products into groups (e.g., TV, Mobile, Refrigerator).
M5	Ledger	Tracks dealer dues, invoices, and payment records.
M6	Media	Stores and manages product images centrally.

4.2 Testing

4.2.1 Test Cases for Unit Testing

Table 4.2 Test Case 001 – Admin Login Module

Project Name: Inventory Control and Management System						
Test Case						
Test Case ID: Login_001				Test Design By: Sakar Dahal		
Module Name: Admin Login Module				Test Design Date: 2025-06-23		
Test Title: Login Admin				Test Executed By: Sakar Dahal		
Description: Test the login module of system				Test execution date: 2025-06-23		
Pre-Conditions: Provided admin with Username and password						
Dependencies:						
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Provide username and password	Username: indrayani Password: password123	Credential can be entered	As Expected,	Pass	
2	Click on Login button		Admin should be able to log in to the system	As Expected, i.e., Admin is redirected to dashboard	Pass	
Post-conditions:						
Admin is validated with username and password and redirected to dashboard.						

4.2.2 Test Cases for System Testing

The test cases for system testing are as follows:

Table 4.3 Test Case 001 Dealer Creation

Project Name: Inventory Control and Management System						
Test Case						
Test Case ID: TC_dealer_creation_001				Test Design By: Sakar Dahal		
Module Name: Dealer Creation				Test Design Date: 2025-6-27		
Test Title: Create New Dealer				Test Executed By: Sakar Dahal		
Description: Test the creation of a new dealer				Test execution date: 2025-6-27		
Pre-Conditions: Admin is logged in						
Dependencies: API connection						
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Navigate to dealers page		Dealers page with all dealers listed	success	Pass	Page loads correctly
2	Click on Add new dealer Button		Opens dealer creation dialog form.	success	Pass	Dialog is responsive
3	Insert dealers details in the form	“company_name”: Baltra, “phone’: 98xxxxx	All the fields are editable and validations are working properly	success	Pass	Only allows valid data insertion
4	Add Dealer		Creation of new dealer	Success	Pass	Dealer created and displayed.
Post-conditions: A new dealer is successfully created.						

Table 4.4 Test Case 002 - Add Product

Project Name: Inventory Control and Management System						
Test Case						
Test Case ID: TC_product_add_002				Test Design By: Sakar Dahal		
Module Name: Project				Test Design Date: 2025-7-01		
Test Title: Create New Product				Test Executed By: Sakar Dahal		
Description: Test the creation of a new Product				Test execution date: 2025-7-01		
Pre-Conditions: Admin is logged in.						
Dependencies: Active API and connection established.						
Step	Test Step	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Navigate to inventory page		Inventory page should open	success	Pass	
2	Click on add new product button		The dialog form should open	success	Pass	
3	Fill product details		Fields should accept input and display validation errors if occurred	success	Pass	
4	Click on add product button		New product should be added and displayed in the table	success	Pass	
Post-conditions:						
The newly created product is displayed in the data table.						

Chapter 5: Conclusion and Future Recommendation

5.1 Conclusion

The Inventory Management System using K-means Clustering for Customer Segmentation effectively addresses the challenges of managing products, stock levels, dealers, and financial transactions within a business. By integrating dealer management, product tracking, stock monitoring, purchase and invoice handling, and customer segmentation, the system provides a comprehensive and centralized solution. The use of modern technologies such as FastAPI, Next.js, and MongoDB ensures real-time data processing, responsive dashboards, and seamless user experience. Furthermore, advanced features like the media center for image management and K-means-based customer segmentation empower businesses to optimize inventory operations, reduce human error, and make informed strategic decisions. Overall, the system enhances operational efficiency, strengthens dealer and customer relationships, and supports accurate financial and inventory reporting.

5.2 Future Enhancement

The following section describes the work that will be implemented with future releases of the software:

- **Integration with E-commerce Platforms:** Connect system with online sales channels to automatically update stock levels and track orders in real time.
- **Advanced Analytics:** Implement predictive analytics to forecast demand, optimize stock levels, and identify trends in customer behavior.
- **Mobile Application:** Develop a mobile app version to allow inventory management and dealer updates on the go.
- **Automated Alerts & Notifications:** Enhance the system to send real-time notifications for low-stock items, pending payments, or order delays via email or SMS.
- **Integration with Accounting Systems:** Link system with accounting software for seamless financial reconciliation and reporting.
- **AI-powered Recommendations:** Utilize AI to suggest optimal stock reorder levels, preferred dealers, or pricing strategies based on historical data.

REFERENCES

- [1] Z. Li, "Development of inventory management system based on multi-Agent technology," *IEEE International Conference on Information Management and Engineering, Chengdu, China*, 2010.
- [2] A. D. a. E. Z. G. Misahuaman, "Web-based systems for inventory control in organizations: A systematic review," *2021 IEEE/ACIS 22nd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, 2021.
- [3] B. Wu, "K-means clustering algorithm and Python implementation," 2018.
- [4] M. N. D. a. A. Priya, "Invoicing and analytics for small and micro manufacturing enterprises," 2020.

APPENDICES

Login Form

Login to your account

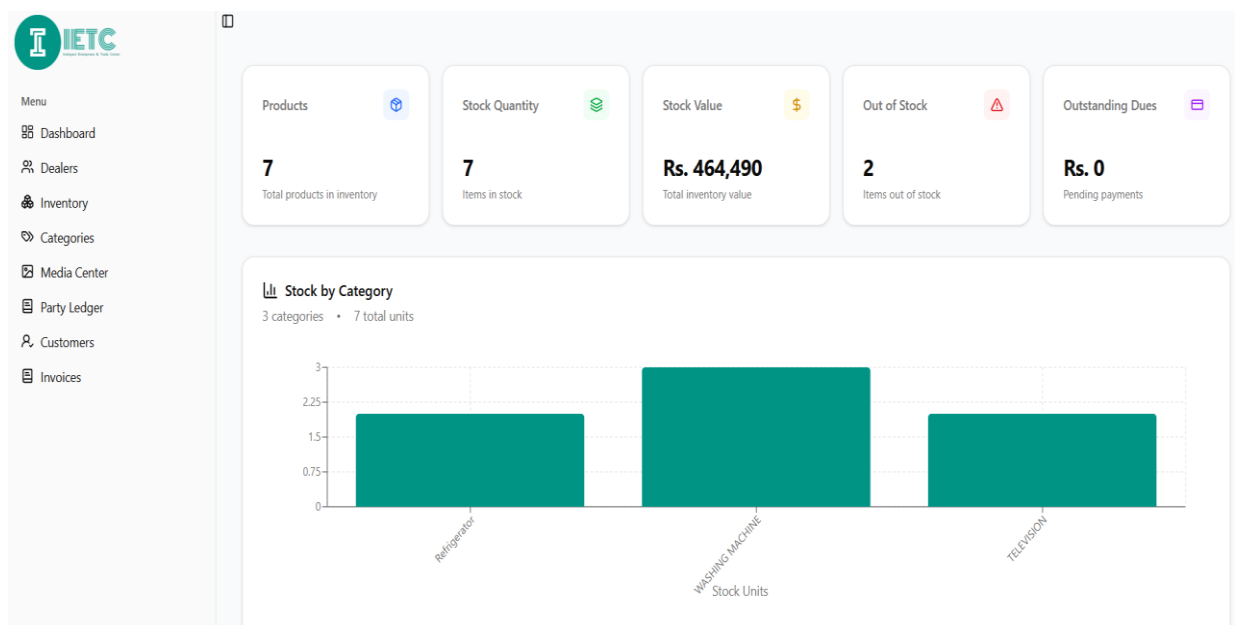
Enter your credentials to login

Username

Password

Login

Dashboard



Dealers Page



Menu

- Dashboard
- Dealers
- Inventory
- Categories
- Media Center
- Party Ledger
- Customers
- Invoices

Logout

Total Dealers: 11

Add New Dealer +

Search dealers...

Columns

☐	Code	Company Name	Contact Person	Phone	Email	Status	Opening Balance	Created At	Actions
☐	DLR003	E.O.L pvt.ltd		9843211483		Active	Rs. 0 (2082-04-19)	8/4/2025	...
☐	DLR004	CG ELECTRONIC PVT LTD		9843211483		Active	-	8/4/2025	...
☐	DLR005	CG impex		9843211483		Active	-	8/4/2025	...
☐	DLR006	NEO INTERNATIONAL		9843211483		Active	-	8/4/2025	...
☐	DLR007	PRASHMA TRADE CONCERN		9843211483		Active	Rs. 45000 (2081-03-03)	8/4/2025	...
☐	DLR008	TRIPURA HARDWARE		9843211483		Active	-	8/4/2025	...
☐	DLR009	NEW GAURA ELECTRICAL		9843211483		Active	-	8/4/2025	...
☐	DLR010	WAWA INTERNATIONAL		9843211483		Active	-	8/4/2025	...
☐	DLR011	JUGAL TRADE LINK		9843211483		Active	-	8/4/2025	...

Inventory Page



Menu

- Dashboard
- Dealers
- Inventory
- Categories
- Media Center
- Party Ledger
- Customers
- Invoices

Total Inventory Items: 7

Add New Product +

Search inventory items...

Columns

☐	Product Code	Image	Name	Model	Category	Dealer	Dealer Price	Selling Price	Stock	Total Sales	Status	Actions
☐	PRD001	N/A	RDEGE 205BXP	THF BR BL	Refrigerator	E.O.L pvt.ltd	Rs 23,500	Rs 25,850	2	1	In Stock	...
☐	PRD002	N/A	RDEGE 205BXP	THF BR WN	Refrigerator	E.O.L pvt.ltd	Rs 23,500	Rs 23,500	0	1	Out Of Stock	...
☐	PRD003	N/A	FRONT LOAD	P110FLG	WASHING MACHINE	E.O.L pvt.ltd	Rs 53,500	Rs 53,500	1	1	In Stock	...
☐	PRD004	N/A	FRONT LOAD	P2210FLG	WASHING MACHINE	E.O.L pvt.ltd	Rs 63,500	Rs 63,500	2	0	In Stock	...
☐	PRD005	N/A	TOP LOADED	PDM8000COBAR	WASHING MACHINE	E.O.L pvt.ltd	Rs 27,500	Rs 27,500	0	1	Out Of Stock	...
☐	PRD006	N/A	T.v	65C735	TELEVISION	CG ELECTRONIC PVT LTD	Rs 168,490	Rs 168,490	1	0	In Stock	...
☐	PRD007	N/A	TV	55V68	TELEVISION	CG ELECTRONIC PVT LTD	Rs 68,500	Rs 68,500	1	0	In Stock	...

Media Center



Menu

- Dashboard
- Dealers
- Inventory
- Categories
- Media Center
- Party Ledger
- Customers
- Invoices

Media Center

2 files in your media library



tcl

5.54 KB Aug 4, 2025, 07:28 AM



cg

3.96 KB Aug 4, 2025, 12:05 PM

Party Ledger PDF

Purchase

SAYANA WOODLAND AND HOME APPILANCE

Ledger Summary

for the period 2082-03-13 to 2082-03-13

Accounts Selection: All Accounts. (All amounts in Rupees)

Date	Document Number	Narration	Debit	Credit	Running Balance
2082-03-13	bill no: 139	purchase	0	129044	129044
2082-03-13	bill no: 140	purchase	0	15064	144108
2082-03-13	bill no: 141	purchase	0	12280	156388
2082-03-13	bill no: 142	purchase	0	41596	197984
2082-03-13	bill no: 143	purchase	0	141288	339272
2082-03-13	bill no: 144	purchase	0	21362	360634
2082-03-13	bill no: 145	purchase	0	4472	365106
Total/Closing Balance			0	365106	365106

Invoice



Menu

- Dashboard
- Dealers
- Inventory
- Categories
- Media Center
- Party Ledger
- Customers
- Invoices

Logout

Back to Invoices

Print Invoice

Invoice #INV0004

Status: Completed

Customer Information

Invoice Details

Items

PRODUCT	QUANTITY	UNIT PRICE	TOTAL
RDEdge 205BXP	1	NPR 28500.00	NPR 28500.00

Subtotal:

NPR 28500.00

Discount:

NPR 0.00

Tax:

NPR 0.00

Total:

NPR 28500.00

Customer Ledger

Dipen Khadka - Customer Ledger

Customer ID: 68ca811b6edb6233e24168f1
Phone: 9868211546

Download

Total Amount Rs. 25,858.01 Across 1 entries	Amount Paid Rs. 0 0 fully paid entries	Outstanding Rs. 25,858.01 1 pending entries	Payment Status 1 Pending
--	---	--	---

From Date
mm/dd/yyyy

To Date
mm/dd/yyyy

Search ledger entries...								Columns
☐	Date	Invoice Number	Amount	Amount Paid	Payment Status	Due Date	Notes	Actions
☐	9/17/2025	INV0007	Rs. 25,858.01	Rs. 0	Pending	10/17/2025	Invoice #INV0007	...

Generated Customer Ledger

Customer Ledger

Customer: Dipen Khadka
Generated on: 9/17/2025

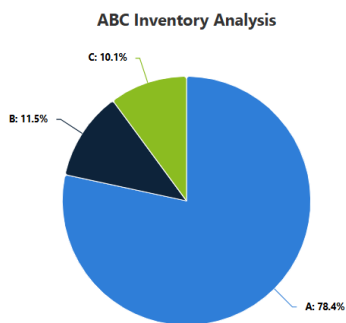
Date	Invoice #	Amount	Amount Paid	Status	Due Date	Notes
9/17/2025	INV0007	Rs. 25,858.01	Rs. 0	PENDING	10/17/2025	Invoice #INV0007
TOTAL		Rs. 25,858.01	Rs. 0			Outstanding: Rs. 25,858.01

Summary:

Total Invoices: 1
Total Amount: Rs. 25,858.01
Total Paid: Rs. 0
Outstanding: Rs. 25,858.01

ABC Analysis

Inventory ABC Analysis



Highcharts.com

Product Breakdown

Product	Value	% of Total	Cumulative %	Category
T.v	168490	36.27%	36.27%	A
FRONT LOAD	127000	27.34%	63.62%	A
TV	68500	14.75%	78.36%	A
FRONT LOAD	53500	11.52%	89.88%	B
RDEdge 205BXP	47000	10.12%	100.00%	C
RDEdge 205BXP	0	0.00%	100.00%	C

Kmeans Clustering

